

Laboratorio #1 — Entrenamiento de Redes Neuronales (MLP)

CC3092 Deep Learning y Sistemas Inteligentes

Ernesto Ascencio 23009

Repositorio: <https://github.com/AscencioSIUU/deepLearning/tree/main/labs/lab1-mlp>

1. Investigación de optimizadores y capas

Capas: `nn.Linear` aplica $y = xW^T + b$ y define el número de neuronas de cada capa vía `out_features`. Las activaciones `nn.ReLU` ($\max(0, x)$), barata, riesgo de neuronas muertas), `nn.LeakyReLU` (deja pasar una fracción del valor negativo, evita neuronas muertas) y `nn.Tanh` (salida en $(-1, 1)$, más propensa a saturación) rompen la linealidad entre capas. `nn.Dropout(p)` apaga aleatoriamente una fracción p de activaciones en entrenamiento para evitar co-adaptación; se desactiva en `model.eval()`. `nn.BatchNorm1d` normaliza activaciones por mini-batch y estabiliza/ acelera el entrenamiento, con parámetros aprendibles de reescalado.

Pérdidas: `nn.MSELoss` penaliza errores cuadráticamente (sensible a outliers); `nn.L1Loss` penaliza proporcionalmente (robusta a outliers, gradiente no suave en 0); `nn.SmoothL1Loss` combina ambas: cuadrática cerca de 0, lineal lejos.

Optimizadores: SGD actualiza $w \leftarrow w - lr * grad$ (+ momentum, + weight_decay como L2 puro); simple pero sensible al lr . Adam adapta el lr por parámetro con promedios móviles del gradiente y su cuadrado; rápido y robusto a la elección inicial de lr . RMSprop usa solo el promedio móvil del cuadrado del gradiente; pensado para problemas no estacionarios. lr controla el tamaño del paso; `weight_decay` penaliza la magnitud de los pesos (regularización L2) para reducir sobreajuste.

2. Preparación de datos (resumen)

California Housing Prices: 20,640 distritos, 9 features + target `median_house_value`. `total_bedrooms` tenía 207 nulos (imputados con la mediana de train); sin duplicados; el target está truncado (capped) en 500,001 USD. Variable categórica `ocean_proximity` codificada con one-hot. Split 60/20/20 train/val/test; imputer y `StandardScaler` ajustados **solo con train**. El conjunto de test no se usó en ninguna decisión de entrenamiento ni de hiperparámetros.

3. Tabla de resultados (15 iteraciones)

#	Arquitectura	Activación	Optim/LR	Batch/Epochs	Regularización	RMSE (val)
1	[64,32]	ReLU	Adam/0.001	32/50	—	68,496
2	[128,64,32]	ReLU	Adam/0.001	32/50	—	65,758
3	[256,128]	ReLU	Adam/0.001	32/50	—	66,518
4	[64,32]	LeakyReLU	Adam/0.001	32/50	—	68,706
5	[64,32]	Tanh	Adam/0.001	32/50	—	234,524
6	[64,32]	ReLU	SGD/0.01	32/50	—	NaN (divergió)
7	[64,32]	ReLU	RMSprop/0.001	32/50	—	68,703

#	Arquitectura	Activación	Optim/LR	Batch/Epochs	Regularización	RMSE (val)
8	[64,32]	ReLU	Adam/0.01	32/50	—	64,924 (mejor)
9	[64,32]	ReLU	Adam/0.0001	32/50	—	195,518
10	[64,32]	ReLU	Adam/0.001	128/50	—	84,418
11	[64,32]	ReLU	Adam/0.001	32/100	—	67,004
12	[64,32]	ReLU	Adam/0.001	32/50	dropout=0.3	69,023
13	[64,32]	ReLU	Adam/0.001	32/50	L2=1e-4	68,502
14	[64,32]	ReLU	Adam/0.001	32/50	L1=1e-5	68,498
15	[128,64]	ReLU	Adam/0.001	32/50	drop=0.2, L2=1e-4, BN	198,223

Modelo final (test, evaluado una única vez): configuración 8 — $MSE \approx 4.10 \times 10^9$, $MAE = \$44,770$, $RMSE = \$64,067$. Consistente con su $RMSE$ de validación (\$64,924), sin señales de fuga de información hacia test.

4. Análisis de resultados

Mayor impacto positivo/negativo: subir lr de Adam de 0.001 a 0.01 dio el mejor $RMSE$ (64,924). El mayor impacto negativo fue usar SGD con $lr=0.01$ y $momentum=0.9$: diverge a NaN en la primera época — el mismo orden de magnitud de lr que ayudó a Adam (que adapta el paso por parámetro) destruyó el entrenamiento con SGD (que no lo hace).

Overfitting/underfitting: overfitting leve en la arquitectura ancha [256,128] (train sigue bajando, val se aplanan). Underfitting claro en lr bajo (0.0001) y en el combo BatchNorm+Dropout+L2: ambos casos muestran train y val bajando juntas pero altas — 50 épocas no bastaron para converger, identificado graficando las curvas de pérdida por época.

Regularización: con este presupuesto de épocas, dropout/L2/L1 individuales no mejoraron sobre el baseline sin regularizar (mismo rango, 68,498–69,023) — el baseline pequeño no estaba sobreajustando, así que regularizarlo no ayudó. La regularización sería más útil en arquitecturas más grandes, que sí mostraron overfitting incipiente.

Batch size / epochs: batch=128 con las mismas épocas empeoró el resultado (menos actualizaciones de pesos por época); epochs=100 sí mejoró respecto a 50 — confirma que varias corridas estaban limitadas por presupuesto de entrenamiento, no por la técnica en sí.

MSE vs. MAE vs. RMSE: MSE amplifica errores grandes (sensible a los distritos con target capado en 500,001); MAE da una lectura “típica” del error sin que esos casos la dominen; RMSE combina unidades de USD con la sensibilidad a outliers de MSE, por eso se usó como criterio de selección.

5. Conclusión: modelo de producción

Arquitectura [128, 64, 32] (mejor $RMSE$ sin regularizar) + Adam con lr intermedio (0.003–0.005, entre el estable 0.001 y el inestable-pero-mejor 0.01) + dropout ligero (0.1–0.2) solo en esa arquitectura más grande + *early stopping* en vez de épocas fijas, para no penalizar configuraciones que solo necesitan más tiempo. Para seguir optimizando: **random search** alrededor de esa zona (arquitectura, lr , dropout) — más eficiente que grid search con variables continuas y una relación no lineal entre lr y optimizador, como se observó aquí. Optimización bayesiana sería el siguiente paso si el costo por corrida creciera; con corridas de segundos, random search ya es suficiente.